

Interactive Video Experience: Gesture-Driven Segmentation Pipeline

Problem

Build an interactive video experience where a user can pause a playing video, tap on any object in the frame, and provide a natural-language instruction (e.g. "open the door"). The system needs to:

1. Identify *which specific object* the user tapped — even when multiple instances of the same class exist in frame (e.g. multiple doors in a hallway).
2. Generate a continuation video where that object responds to the instruction.
3. Feel responsive enough that the interaction loop doesn't break immersion.

This is not a pre-authored branching experience. Outcomes are dynamic and driven by user intent.

Reference: Apple's Subject Lifting UX

Apple's "tap and hold to lift the subject" feature sets the bar for perceived speed. Under the hood:

- Runs on the Apple Neural Engine via the Vision framework (`VNGenerateForegroundInstanceMaskRequest`, iOS 17+).
- Combines saliency detection with instance segmentation, plus a matting refinement step for clean edges.
- The semantic label ("this is a dog") is computed *lazily*, after the mask is already displayed — the shimmer animation is the acknowledgment, not the resolution.

Key UX principle to steal: decouple acknowledgment from resolution. Show the user the system heard them before it's done thinking.

Pipeline

Inputs at interaction time

- `frame_N` : the paused video frame (RGB image).
- `(x, y)` : tap coordinates on that frame.
- `instruction` : the user's natural-language prompt (e.g. "open the door").

Steps

1. Parse instruction → extract target noun phrase + action verb.

- Small/fast LLM call. Pulls `target = "door"`, `action = "open"`.
- Cheap, ~100ms.

2. Segment with SAM 3.

- Pass `frame_N` + point prompt `(x, y)` + concept prompt `"door"`.
- SAM 3 natively supports text/concept prompts alongside point prompts, which solves the multi-instance disambiguation problem in a single model call. The point selects *which* door; the concept ensures the mask covers the door panel and not the surrounding wall.
- Returns a binary mask of the tapped object.
- Hosted GPU inference, ~100–300ms.

3. Build generation payload.

- `frame_N` → first-frame conditioning for image-to-video.
- `mask` → spatial control signal (region of interest, inpainting mask, or ControlNet-style conditioning depending on the video model).
- `action` → text prompt driving motion ("door swings open").

4. Generate continuation video.

- Image-to-video model with regional control (Veo, Runway, Kling, or open alternatives).
- This is the latency bottleneck: seconds to tens of seconds.
- Hide latency with diegetic pacing — darkness, sound design, camera push-in. Horror genre is forgiving here.

5. Stitch.

- `frame_N` is the last frame of clip A and the first frame of clip B.

- Transition is invisible if the generation model respects the conditioning.

Why SAM 3 Specifically

The multi-instance problem (multiple doors in frame) is the exact case where SAM 2 would force a separate VLM disambiguation pass. SAM 3's Promptable Concept Segmentation makes the concept prompt native to the segmentation call:

- Tap + "door" → mask of *the door you tapped*.
- No separate "what is this" step needed.
- Released by Meta on November 19, 2025; weights and code are open under the SAM license.
- Architecture: 848M-parameter model with a shared vision encoder feeding a DETR-based detector (conditioned on text, geometry, and image exemplars) and a SAM 2-derived tracker.

Latency Tiers (Apple-style staggering)

Tier	Budget	What happens	What the user sees
Instant	<100ms	Tap registered, mask request fired	Visual feedback on the tap point (shimmer, glow)
Fast	200–800ms	SAM 3 returns mask, instruction parsed	Object outlined, "thinking" affordance on the prompt
Slow	seconds+	Video generation	Pacing beat — darkness, sound, push-in

Open Architectural Questions

- **Action-verb-conditioned mask adjustment.** "Open the door" wants the door panel. "Burn the door" might want the panel + frame. For v1, the panel-only mask is likely good enough — the video model can extrapolate. Worth revisiting if generation quality suffers.
- **Continuous vs on-tap segmentation.** Running SAM 3 continuously on the playing video (tracking salient objects in the background) would make the tap-to-mask step a lookup rather than a computation. Trade-off: GPU cost vs. latency. Probably not worth it for a paused-video interaction model.

- **Pre-labeling interactables.** As objects enter frame, a background VLM could tag them ("door," "painting," "light switch"). Pairs well with continuous segmentation. Also probably not necessary if SAM 3's concept prompt is reliable.
- **Hosted vs self-hosted inference.** SAM 3 needs GPU. Hosted endpoint is simpler for v1; self-hosting matters once usage scales.

Stack Summary

Layer	Choice
Instruction parsing	Small LLM (local or fast API)
Segmentation	SAM 3 (point + concept prompt)
Video generation	Image-to-video model with regional/mask conditioning
Orchestration	Pause → capture → parse → segment → generate → stitch

Next Steps

- Prototype the segmentation step in isolation: still images, tap + concept prompt, validate that multi-instance disambiguation works on a hallway-with-multiple-doors test set.
- Identify the video generation model with the best mask/regional conditioning support.
- Build the staggered-feedback UX shell so latency tiers can be tuned independently.